



Phage
SECURITY

SECURITY REVIEW · PREPARED FOR

Robin Markets

A Polygon split vault earning
Polymarket holder rewards
on hedged outcome pairs

DATE

03.09.2026

CONDUCTED BY

Pyro, Lead Security Researcher
MarketerKA, Security Researcher

COMMIT

49bd9b1

REMEDIATION

a8d94d6

01 ABOUT

Phage Security

Phage Security is a team of highly skilled smart-contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike.

With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

PREVIOUS WORK

github.com/phage-security

REACH OUT

Telegram · @Pyro3b

WHAT AN AUDIT IS

An audit is a time, resource, and expertise-bound effort in which trained researchers evaluate smart contracts using a combination of manual and automated techniques to identify as many vulnerabilities as possible.

An audit can reveal the presence of vulnerabilities, but it cannot guarantee their absence. This report is a point-in-time assessment of the code at the commits named on the cover. It is not a warranty that the system is secure.

02 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	ACKNOWLEDGED
4	1	3

SEVERITY DISTRIBUTION



CLIENT	REPOSITORY	METHODS
Robin Markets	github.com/robin-markets/split-vault-contracts	Manual review

CONTRACTS IN SCOPE

src/RobinSplitVault.sol	src/RobinPushDepositRouter.sol
-------------------------	--------------------------------

TIMELINE · SEPTEMBER 2026

Audit · 2 days		Remediation · 2 days	
KICK-OFF	END OF AUDIT	REMEDiations START	REMEDiations END
30.08.2026	31.08.2026	02.09.2026	03.09.2026

STARTING COMMIT HASH	49bd9b1c3598a2b09752df38e4e9c344c9938500
FINAL COMMIT HASH	a8d94d676ced02f8dc0d96308c06d088bb27a7b6
PUBLIC FINAL COMMIT HASH	2446b179166fd660ece9fedd3ab5c39dfc393bc7

03 PROJECT SUMMARY

What Robin Markets' split vault is

The Robin Markets Split Vault lets users deposit USDC.e, splits every dollar into a YES and NO pair on a Polymarket rewards market, and pays out the daily holder rewards as vault yield.

A YES and NO pair always merges back to exactly 1 USDC.e, before and after resolution, so the position is fully hedged and withdrawals are instant and at par. The split vault is one component of the wider Robin Markets platform: it replaces the retired Yearn vault as the yield source of the staking vault, and the manager can rotate markets when one stops paying.

Rewards arrive as PolyUSD roughly every 24 hours and stay invisible to the share price until the permissionless `compound()` unwraps them to USDC.e and folds them into a linear vesting schedule, so each payout drips out at roughly the rate it was earned and cannot be sniped. A stateless router lets approve-blocked contract wallets enter by batching a transfer with a deposit call.

SECURITY POSTURE

BEFORE REMEDIATION

49bd9b1

Book close on raw share count

M-01 · L-03

The empty-vault gate compared `totalSupply()` against a constant instead of asking what the shares redeem to. A compound-grown share price let the books close on a funded holder, and shares parked on the vault itself hold the gate open forever.

Reward recognition under offramp downtime

L-01 · L-02

A failing PolyUSD offramp freezes the accrual clock, and the vault cannot free the stuck leg itself. Once the offramp recovers, daily compounds re-date the running schedule and a 14-day backlog pays out in 10.

AFTER REMEDIATION

a8d94d6

One of four findings fixed and verified. The book-close and recognition gates now ask what the remaining shares collectively redeem to, `convertToAssets(totalSupply()) = 0`, instead of counting raw shares, so an exit can no longer zero a funded holder's claim. During the fix review the team also added zero-share guards to both ERC-4626 hooks and events for the two previously silent accounting resets.

The three acknowledged findings sit on the reward leg or need a self-inflicted share transfer: deposits and withdrawals stay at par in every scenario, and the exposure is confined to yield timing and ownerless residue. The vesting compression after a backlog has no simple patch, a proper fix means reworking how the schedule tracks tranches, and the team accepts that behavior for now.

04 RECOMMENDATIONS

Beyond this audit

Four measures that extend the protection of this review after launch, independent of any individual finding.

R-01	Bug bounty	Stand up a public bounty on Immunefi or HackenProof with the top tier covering fund loss.
R-02	Invariant monitoring	Run an off-chain watcher that reconciles vault state against the chain and pages on divergence. Watch that <code>idle USDC.e</code> plus <code>pairedAssets()</code> never drops below <code>trackedAssets</code> , that <code>lastRecognitionTime</code> does not go stale past the keeper cadence, that <code>UnwrapFailed</code> does not repeat across compounds, and that the vault's own share balance stays zero.
R-03	Offramp runbook	The vault cannot free a stuck PolyUSD leg on its own. Agree with Polymarket in advance on the recovery path for a paused or replaced offramp, including the role grant a <code>set0fframp</code> replacement needs.
R-04	Key hygiene	The timelock controller and the owner Safe gate upgrades, the offramp, and market rotation. Keep both on hardware signers behind a multisig with a documented rotation and incident runbook.

05 **CONTRACTS & ROLES**

Core contracts

CONTRACT	ROLE
<code>src/RobinSplitVault.sol</code>	ERC-4626 vault splitting deposited USDC.e into YES and NO pairs on the manager-set Polymarket market, recognizing rewards through the permissionless <code>compound()</code> behind a linear vesting schedule, UUPS upgradeable behind <code>TIMELOCKED_ROLE</code> .
<code>src/RobinPushDepositRouter.sol</code>	Stateless, unowned periphery letting approve-blocked wallets enter the vault by atomically batching a transfer with <code>depositFor</code> , stray balances claimable by anyone via <code>rescue</code> .

Polygon, Solidity 0.8.31. `RobinSplitVault` is UUPS upgradeable, `RobinPushDepositRouter` is stateless and unowned.

PRIVILEGED ROLES

DEFAULT_ADMIN_ROLE. Grants and revokes the admin and manager roles, held by the owner Gnosis Safe.

DEFAULT_MANAGER_ROLE. Rotates the active market with `setMarket` and moves the vesting floor within 8 hours to 30 days, held by the owner Gnosis Safe.

TIMELOCKED_ROLE. Self-administered, gates UUPS upgrades and `setOfframp`, held by `RobinTimeLockController`.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
M-01	Books are cleared while a holder still has value	MEDIUM	● FIXED
L-01	<code>compound()</code> moves the vesting end backwards after a backlog	LOW	● ACKNOWLEDGED
L-02	The vault cannot recover from an offramp that stops accepting its asset	LOW	● ACKNOWLEDGED
L-03	Shares sent to the vault itself can never be removed	LOW	● ACKNOWLEDGED

● MEDIUM SEVERITY · 1 FINDING

M-01 MEDIUM FIXED

Books are cleared while a holder still has value

Once everyone has left, the vault clears its own accounting so leftover yield becomes invisible again and the next depositor can pick it up. Whether everyone has left is decided by counting shares, not by checking whether anyone still holds value:

```
if (totalSupply() < 10 ** _decimalsOffset()) {
    $.trackedAssets = 0;
    $.vestingAmount = 0;
```

Those two agree only at the genesis rate. As `compound()` folds yield in, the same money buys fewer shares, so a funded depositor lands under the threshold and the branch fires on him. The same constant gates recognition in `compound()`, blocking self-rescue, and the comment above the branch promises the opposite: "Empty" means less than 1 asset wei of claims, forfeiting at most about 1 wei.

1. The staking vault holds 1,000,000 USDC.e and one 89.04 USDC.e reward is compounded
2. It exits mid-vest, leaving 44.52 USDC.e owned by nobody
3. An attacker deposits 1 wei, minting exactly the threshold share count
4. Anyone compounds, the 44.52 attaches to those shares, and a depositor adds 20 USDC.e for a claim of 19.999992
5. The attacker redeems in the same block: his exit closes the books on the funded depositor and takes 43.39 USDC.e

	BEFORE THE ATTACKER EXITS	AFTER
total shares	1,898,461	898,461
accounted backing	64.5 USDC.e	0
victim's claim	19.999992 USDC.e	0
USDC.e in the contract	64.5	64.5

The money stays, the claim disappears. A deposit is exposed while it is smaller than about half the leftover, the leftover grows with keeper downtime, and every turnover of holders recreates the state.

RECOMMENDATION

Gate on what the remaining shares are worth rather than how many there are, ignoring any that sit on the vault itself, and sweep the leftover on close instead of zeroing it. The gate alone is not enough, because the leftover then stays on the books and stays cheap to buy up.

● LOW SEVERITY · 3 FINDINGS

L-01

LOW

ACKNOWLEDGED

compound() moves the vesting end backwards after a backlog

compound() keeps one vesting schedule. Every call folds the new tranche into what is left of the old one and rewrites all three fields:

```
uint256 remainder = _unvestedProfit($);
uint256 remaining = remainder > 0 ? $.vestingEnd - nowStamp : 0;
uint256 numerator = remainder * remaining + recognized * duration;
uint256 denominator = remainder + recognized;
uint256 blended = duration < remaining ? Math.ceilDiv(numerator, denominator) : numerator /
denominator;
$.trackedAssets = backing;
$.vestingAmount = remainder + recognized;
$.vestingStart = nowStamp;
$.vestingEnd = nowStamp + uint64(blended);
```

blended is the amount-weighted mean of remaining and duration, so it always lands between the two. vestingStart and vestingEnd then re-date the whole pool, including the part that was already scheduled to end later. Subtracting the old end gives recognized * (duration - remaining) / (remainder + recognized), negative whenever duration < remaining. That is the state after a backlog: duration is the time since lastRecognitionTime, floored at baseVestingDuration, so a daily keeper pins it at 24 hours while remaining is still two weeks out. Every daily compound pulls the end in until the backlog has drained. The natspec states the opposite in three places, dead time "can only stretch a tranche, never compress a backlog".

1. The vault holds 1,000,000 USDC.e at a 24 hour base vesting duration, Polymarket pays 3.25% APR, 89.041095 USDC.e per day
2. The offramp is down for 14 days, lastRecognitionTime stays frozen, which is what the code intends
3. It recovers on day 14 and the keeper compounds: 1,246 USDC.e is scheduled over 14 days, ending day 28, which is correct
4. Daily compounds resume, each recognizing about 89 USDC.e with a duration of 24 hours against a remaining of nearly two weeks
5. A second depositor puts in 1,000,000 USDC.e on day 19 and redeems on day 24, collecting 710 USDC.e where the fair five-day yield is about 445

COMPOUND	vestingEnd	DRIP VS EARN RATE
day 14	day 28.000	1.00x
day 17	day 25.786	1.54x
day 21	day 24.226	2.99x
day 23	day 24.052	4.70x

That is 5.18% APR for the second depositor against a 3.25% program rate, and the staking vault settles about 711 USDC.e short, 0.07% of TVL. The window is open only while a backlog drains, the position is redeemable at par throughout, and no role is required.

RECOMMENDATION

Given the current mechanics we are not able to propose a simple fix. Resolving this properly would require reworking how the schedule tracks tranches.

L-02

LOW

ACKNOWLEDGED

The vault cannot recover from an offramp that stops accepting its asset

Rewards arrive in a token the vault cannot hold as backing, so it converts them through a Polymarket contract and only moves the accrual clock when that works:

```
try this.unwrapRewards(0) {
  unwrapped = true;
} catch {
  emit UnwrapFailed();
}
...
if (unwrapped) $.lastRecognitionTime = nowStamp;
```

The conversion is paused per asset, and the flag already sits on native USDC while USDC.e is open. Set on USDC.e, the clock stops for good, and both documented remedies fail: `unwrapRewards` offers partial draws, but `compound()` asks for zero and zero means the whole balance, and a `setOfframp` replacement cannot release the underlying until PolyUSD's owner grants it a role.

On a Polygon fork with 1,000 USDC.e and 10 USDC tranches: day one pays 0.333334 instead of 10.000000, the clock is unchanged over 165 days, and once the cohort exits and the books close 60 USDC sit with no claimant.

The trigger belongs to Polymarket, not to a vault user or role, and deposits and withdrawals stay at par, so the loss is confined to reward yield.

RECOMMENDATION

Let the clock advance even when the conversion fails, capping the gap at the maximum vesting duration, so one unconvertible balance cannot pin the schedule for every holder. Add a timelocked way to move the reward token out, or narrow the comment promising the leg heals once the offramp recovers, since the vault cannot perform that recovery itself.

L-03

LOW

ACKNOWLEDGED

Shares sent to the vault itself can never be removed

The vault clears its own accounting once every holder has left, so leftover yield returns to circulation. Nothing stops shares being parked at the vault's own address, where they can never be redeemed: the contract has no burn function, it never approves anyone to spend its own shares, and `redeem(shares, x, vault)` reverts on the allowance check for every caller.

The share count then never falls below the threshold, so the book-close in `_withdraw` and the recognition gate in `compound()` both die, contradicting the comment that promises the close. Three paths reach the state, and a mistake is enough for any:

1. `deposit(1, address(vault))`, since OZ's `_mint` rejects only the zero address
2. `RobinPushDepositRouter::depositFor` with the vault as receiver, free out of a stray balance
3. a plain `transfer` of shares already held

Measured result: 89.041096 USDC.e of claim on unredeemable shares, no profit for anyone, undone only by an upgrade.

RECOMMENDATION

Guard the hook every path passes through:

```
+/// @dev The vault can never spend its own shares, so nothing may put shares there.
+function _update(address from, address to, uint256 value) internal override {
+    if (to == address(this)) revert UnsolicitedTransfer(to);
+    super._update(from, to, value);
+}
```